Informe final: productor y consumidores con semáforo

Proyecto final de Sistemas Operativos 1 · Redis, MySQL, Python y Docker Compose

Objetivo y arquitectura

El sistema simula transacciones bancarias mediante cinco contenedores. Un productor inserta filas con estado **PENDIENTE** en MySQL. Dos consumidores buscan la fila pendiente más antigua, esperan entre uno y tres segundos y la cambian a **PROCESADO**. Redis guarda un semáforo distribuido para coordinar a los consumidores. MySQL también conserva cada intento en `intentos_procesamiento`, lo que permite observar escrituras repetidas.

Problema y sección crítica

La sección crítica abarca la búsqueda de una transacción pendiente, la simulación de su procesamiento y la actualización de su estado y metadatos. Si ambos consumidores leen la misma fila antes de que alguno la actualice, ambos pueden trabajar sobre ella. La fila final solo muestra un consumidor, de modo que el resultado visible puede ocultar el trabajo duplicado y la sobrescritura de `procesado_por`.

Semáforo distribuido en Redis

Cada consumidor intenta `SET semaforo:transacciones <token> NX EX 10`. `NX` hace que solo el primero cree la clave; el token combina el nombre del consumidor con un identificador único; `EX 10` evita que un fallo deje el recurso bloqueado indefinidamente. Mientras procesa, el consumidor renueva el vencimiento si todavía es dueño. Para liberar, un script Lua compara el token y elimina la clave en una sola operación atómica. Así, un consumidor no elimina el bloqueo que otro adquirió después de una expiración.

Prueba y resultados observados

Se ejecutaron dos corridas independientes con un volumen MySQL limpio en cada una. El productor se detuvo después de crear más de 20 transacciones; los consumidores continuaron hasta dejar cero pendientes. Los resultados se sustentan en los archivos de `trazas/`.

Medida	Con semáforo	Sin semáforo
Transacciones creadas	26	29
Transacciones procesadas	26	29
Pendientes al cierre	0	0
Intentos duplicados registrados	0	26

Con semáforo, los logs muestran adquisición, procesamiento y liberación alternados entre consumidores; cada transacción tuvo un solo intento. Sin semáforo, ambos consumidores llegaron a seleccionar el mismo ID antes de actualizarlo. La tabla de intentos registró 26 segundas escrituras sobre transacciones que ya estaban procesadas.

Condición de carrera y alcance

La carrera es de tipo lectura, espera y escritura: el resultado depende del orden temporal de dos consumidores. En el modo de demostración sin semáforo, la actualización deliberadamente no exige que la fila continúe `PENDIENTE`; esto hace visible la sobrescritura y registra el segundo intento como `DUPLICADO`. En el modo normal, además del semáforo, la sentencia `UPDATE` exige `estado='PENDIENTE'` como protección adicional. El registro de intentos permite distinguir entre transacciones únicas y operaciones repetidas.

Conclusiones

La coordinación en Redis evitó que los dos consumidores ejecutaran simultáneamente la sección crítica durante la prueba. La liberación condicionada al token evita borrar la concesión de otro consumidor. La comparación real mostró cero duplicados con sincronización y 26 intentos duplicados sin ella. El TTL y su renovación ayudan ante fallos o pausas, aunque una instalación bancaria real requeriría garantías adicionales de consistencia y recuperación que exceden esta simulación académica.

> **Nota sobre el enunciado:** el documento recibido menciona preguntas de una sección 9 que no fue proporcionada. Este informe responde solo a los requisitos disponibles; no atribuye preguntas inexistentes al documento original.